

Foundations of Machine Learning

Assignment 1 Report

CS25S023

September 21, 2025

1. Introduction

In this project, I applied Principal Component Analysis (PCA) for dimensionality reduction and K-Means Clustering for grouping patterns.

The experiments were conducted on two datasets:

1. **dataset1:** This dataset contains 1,000 datapoints in $\mathbb{R}^2$. This dataset is non-linear in nature. We have tried running Standard PCA. As well as, we have used this dataset for Kernel PCA analysis.
2. **dataset2:** This dataset, containing 1,000 datapoints in $\mathbb{R}^2$, is being used for clustering analysis. We have used this dataset for analysis of K-means Clustering. And as this is non-linear in nature, We are also going to use this for Spectral Clustering analysis

Definitions

- **Principal Component Analysis (PCA)** PCA is a statistical method for **linear dimensionality reduction**. It transforms a set of possibly correlated variables into a smaller set of linearly uncorrelated variables known as **principal components**. The first principal component accounts for the largest possible variance in the data, with each subsequent component accounting for the next largest variance while being orthogonal to the previous components. This technique is highly effective for simplifying high-dimensional datasets while preserving the most significant information.
- **Kernel PCA** Kernel PCA is an extension of standard PCA designed for **non-linear dimensionality reduction**. Unlike standard PCA, which operates on the original feature space, Kernel PCA first uses a **kernel function** (the "kernel trick") to implicitly map the data into a higher-dimensional feature space. In this new space, the data may become linearly separable, allowing a standard PCA to be performed. This approach enables Kernel PCA to capture and represent complex, non-linear structures in the data that standard PCA would otherwise miss.
- **K-Means Clustering** K-Means is an unsupervised machine learning algorithm that partitions data into a predefined number of clusters (K). The algorithm minimizes the variance within each cluster by iteratively assigning data points to the nearest cluster center and updating the centers based on the mean of the assigned points. It is widely used for pattern recognition and grouping similar data points.
- **Kernel Clustering** Kernel clustering refers to a category of clustering algorithms that use the **kernel trick** to find non-linear clusters in a dataset. A common example is **Kernel K-Means**, which is a non-linear extension of the standard K-Means algorithm. This method can identify clusters with arbitrary shapes by operating in a transformed, higher-dimensional feature space. Instead of explicitly computing coordinates in that space, it relies on the kernel function to calculate similarity, allowing it to reveal hidden patterns that a linear clustering algorithm might miss.

Libraries Used

The following Python libraries were utilized for data preprocessing, dimensionality reduction, clustering, visualization, and performance evaluation:

- **Numpy:** For numerical computations and array manipulations.
- **Pandas:** For loading and processing datasets.
- **Matplotlib:** For data visualization and cluster plotting.

Assignment Objectives

Objectives for Question 1

The primary objective of Question 1 is to implement and compare two dimensionality reduction techniques: **Principal Component Analysis (PCA)** and **Kernel PCA** on a dataset of 1000 points in $\mathbb{R}^2$. The task is broken down into three parts:

- **Part (a):** Implement a PCA algorithm from scratch and determine the variance explained by each principal component. The only allowed library is for computing eigenvectors and eigenvalues.
- **Part (b):** Implement the Kernel PCA algorithm from scratch using various kernels [cite: 27]. For each kernel, you must plot the projection of the data onto the top-2 principal components. Specifically, for the RBF kernel, a separate plot is required for each value of σ that you use.
- **Part (c):** Based on the outputs from the previous parts, you must select the kernel you believe is best suited for the dataset and provide a clear explanation for your choice.

Objectives for Question 2

The objective of Question 2 is to apply and analyze two clustering algorithms: **K-Means Clustering** and **Spectral Clustering**, on a dataset of 1000 data points in $\mathbb{R}^2$. The question is divided into four parts:

- **Part (a):** Write a piece of code to run the K-Means algorithm with $k = 4$. You must explore five different random initializations, plotting the error function with respect to iterations and the resulting clusters for each case.
- **Part (b):** Fix one random initialization and run the K-Means algorithm for $K = \{2, 3, 4, 5\}$. For each K value, you need to plot the Voronoi regions associated with the cluster centers.
- **Part (c):** Run a spectral clustering algorithm with $k = 4$, which uses the spectral relaxation of K-means via Kernel PCA. You must choose an appropriate kernel, plot the resulting clusters, and explain your choice based on the output.
- **Part (d):** Instead of the standard spectral clustering method, use an alternative approach for assigning data points to clusters. You will assign a data point i to a cluster l based on the maximum value in its corresponding eigenvector, using the formula $l = \arg \max_{j=1, \dots, k} v_{ji}$. You are required to analyze how this mapping performs for the dataset and explain your insights.

2. Mathematical Formulation

Principal Component Analysis (PCA)

1. Data Centering

To center the dataset, subtract the mean of each feature:

$$X_{\text{centered}} = X - \mu$$

where:

- $X \in \mathbb{R}^{n \times d}$ is the original dataset with n samples and d features.
- $\mu \in \mathbb{R}^d$ is the mean vector.

2. Covariance Matrix Calculation

The covariance matrix captures feature relationships:

$$C = \frac{1}{n} X_{\text{centered}}^T X_{\text{centered}}$$

3. Eigen Decomposition

Principal components are determined via eigen decomposition:

$$C v_i = \lambda_i v_i$$

where:

- λ_i is the eigenvalue representing the variance explained by the i-th component.
- v_i is the eigenvector representing the principal component direction.

4. Explained Variance

The proportion of variance explained by the i-th principal component is:

$$\text{Explained Variance Ratio}_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$

5. Dimensionality Reduction

Project data onto the subspace spanned by the top k principal components:

$$Z = X_{\text{centered}} V_k$$

where $V_k = [v_1, v_2, \dots, v_k] \in \mathbb{R}^{d \times k}$

Algorithm: PCA

1. Input: Dataset X, number of components k.
2. Compute μ and center the data: $X_{\text{centered}} = X - \mu$.
3. Compute covariance matrix: C.
4. Perform eigen decomposition to get (λ_i, v_i) .
5. Select the top k eigenvectors to form V_k .
6. Project centered data: $Z = X_{\text{centered}} V_k$.

Kernel Principal Component Analysis (Kernel PCA)

1. The Kernel Trick and Matrix Calculation

Unlike standard PCA, Kernel PCA works in a high-dimensional feature space where data may be linearly separable. This is achieved without explicitly computing the coordinates in this space. Instead, a **kernel function** $\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ is used to compute the dot products between data points. This allows us to construct the **Kernel Matrix** $K \in \mathbb{R}^{n \times n}$, where each element is $K_{ij} = \kappa(x_i, x_j)$.

$$K = [\kappa(x_i, x_j)]_{i,j=1}^n$$

where x_i, x_j are data points.

2. Centering the Kernel Matrix

To ensure the data is centered in the feature space, we must center the kernel matrix. This is done using the following formula:

$$\tilde{K} = K - \mathbf{1}_n K - K \mathbf{1}_n + \mathbf{1}_n K \mathbf{1}_n$$

where $\mathbf{1}_n$ is the $n \times n$ matrix with all entries equal to $1/n$.

3. Eigen Decomposition

The principal components are found by performing an eigen decomposition on the centered kernel matrix:

$$\tilde{K} \alpha_i = \lambda_i n \alpha_i$$

where:

- λ_i is the eigenvalue, representing the variance of the i -th principal component.
- α_i is the eigenvector corresponding to λ_i .

4. Dimensionality Reduction

To project a new data point x onto the principal components, we use the eigenvectors and the kernel function:

$$Z_j = \sum_{i=1}^n \alpha_{ji} \kappa(x_i, x)$$

where α_{ji} is the i -th component of the j -th eigenvector α_j and Z_j is the projection on the j -th principal component.

Common Kernel Functions

1. **Polynomial Kernel:** Maps data into a feature space of polynomials of degree p .

$$\kappa(x_i, x_j) = (1 + x_i^T x_j)^p$$

2. **Radial Basis Function (RBF) Kernel:** Maps data into an infinite-dimensional space.

$$\kappa(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

where $\sigma > 0$ controls the kernel's bandwidth.

Algorithm: Kernel PCA

1. Input: Dataset X , kernel function κ , number of components k .
2. Compute the kernel matrix: K .
3. Center the kernel matrix: $\tilde{K}$.
4. Perform eigen decomposition to get (λ_i, α_i) .
5. Select the top k eigenvectors to form $\mathbf{A}_k = [\alpha_1, \alpha_2, \dots, \alpha_k]$.
6. Project new data points onto the principal components.

K-Means Clustering

K-Means is a centroid-based clustering algorithm that partitions data into a predefined number of clusters, aiming to minimize the distance between data points and their assigned cluster centroid.

1. Objective Function

The goal is to minimize the within-cluster sum of squared distances (WCSS) to find the optimal cluster partition.

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

where C_i is the set of points in cluster i and μ_i is the centroid of that cluster.

2. Algorithm Steps

- (a) **Initialization:** Randomly select k initial centroids $\{\mu_1, \dots, \mu_k\}$ from the dataset.
- (b) **Assignment Step:** Assign each data point x_j to its nearest centroid based on Euclidean distance: $C(j) = \arg \min_i \|x_j - \mu_i\|^2$.
- (c) **Update Step:** Compute the new centroids as the mean of all points assigned to each cluster: $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$.
- (d) **Convergence Check:** Repeat the assignment and update steps until the centroids no longer change or a maximum number of iterations is reached.

Algorithm: K-Means

1. Input: Dataset X, number of clusters k.
2. Initialize k cluster centroids randomly.
3. Repeat until convergence:
 - Assign each data point to its nearest centroid.
 - Update each centroid to the mean of its assigned points.

Spectral Clustering

Spectral clustering is a method for clustering data that are not linearly separable. It transforms the problem into finding a partition of the data in an embedded space, by using the eigenvalues and eigenvectors of a similarity matrix.

1. Similarity Graph Construction

The first step is to model the data as a graph $G = (V, E)$, where each data point is a vertex $v \in V$. The edges E represent the similarity between points. A common approach is to use an affinity or similarity matrix W , where W_{ij} is a measure of similarity between points x_i and x_j (e.g., using an RBF kernel).

2. Laplacian Matrix Calculation

The **Laplacian matrix** is crucial as it reveals the structure of the graph. It is defined as:

$$L = D - W$$

where W is the similarity matrix and D is the diagonal degree matrix, with $D_{ii} = \sum_j W_{ij}$. The Laplacian matrix effectively acts as a discrete operator that reveals the underlying connectivity of the data. Eigenvectors of the Laplacian, particularly those corresponding to the smallest eigenvalues, capture the low-dimensional structure of the data and are used to embed the points in a new space where clusters are more easily separable.

3. Eigen Decomposition

Find the first k eigenvectors of the Laplacian matrix corresponding to the k smallest eigenvalues. Let these eigenvectors form a matrix $U \in \mathbb{R}^{n \times k}$. This matrix U provides a new, low-dimensional representation of the data where the cluster structure is more apparent.

4. Clustering

Treat each row of the matrix U as a new data point in a k -dimensional space. Now, apply a standard clustering algorithm, such as **K-Means**, to these new data points to obtain the final clusters.

Algorithm: Spectral Clustering

1. Input: Dataset X , number of clusters k , similarity function.
2. Construct a similarity matrix W .
3. Compute the degree matrix D .
4. Calculate the Laplacian matrix $L = D - W$.
5. Find the first k eigenvectors of L .
6. Form a new data matrix with these eigenvectors as columns.
7. Apply K-Means to the new data matrix to get the final clusters.

3. Experiments and Results

1.1: Standard PCA on Dataset1

Implementation

PCA was applied to Dataset 1, which consists of 1000 data points in $\mathbb{R}^2$. The data was centered by subtracting the mean for each of the two features. Eigen decomposition was performed on the covariance matrix to obtain the principal components.

Results and Analysis

The cumulative explained variance was plotted to determine the optimal number of components to retain.

- Explained variance by PC1: 0.6539
- Explained variance by PC2: 0.3461

It was observed that the first principal component (PC1) explains approximately 65.39% of the variance, while the second principal component (PC2) explains the remaining 34.61%

Data Visualization

The PCA algorithm transforms the data by rotating and compressing it to align with the directions of maximum variance. In the provided plot, the original data points form distinct, intertwined spirals. The PCA projection shows that the algorithm has stretched and rotated the spirals. While it preserves the overall spread of the data, PCA is a linear method, and thus it cannot separate the non-linear, intertwined spirals in a meaningful way. The transformed data still shows a similar complex, intertwined pattern, indicating that a non-linear method would be more effective for this dataset.

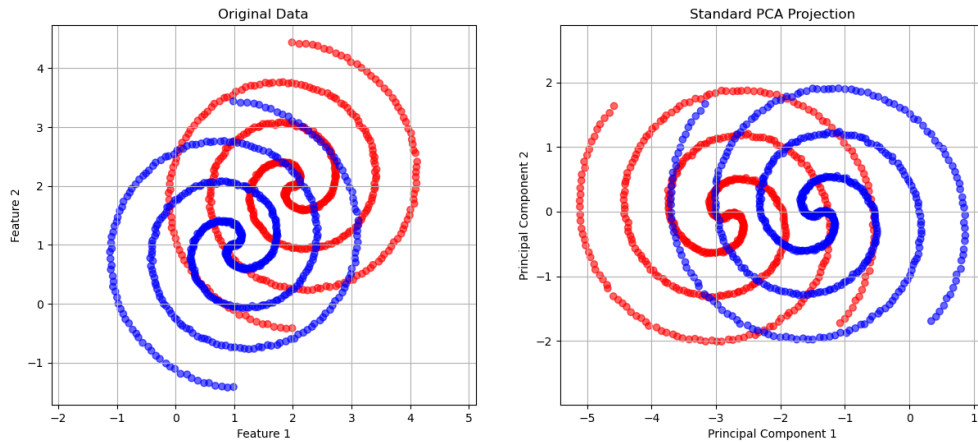


Figure 1: Original And Standard PCA Plots

1.2: Kernel PCA on Dataset1

Implementation

Kernel PCA was applied to Dataset 1, which consists of 1000 data points in $\mathbb{R}^2$. Various kernel functions, including the **Polynomial Kernel** and the **Radial Basis Function (RBF) Kernel**, were used to explore the non-linear structure of the dataset. The kernel matrix was computed for each kernel, centered, and then eigen decomposition was performed to obtain the principal components.

Visualization

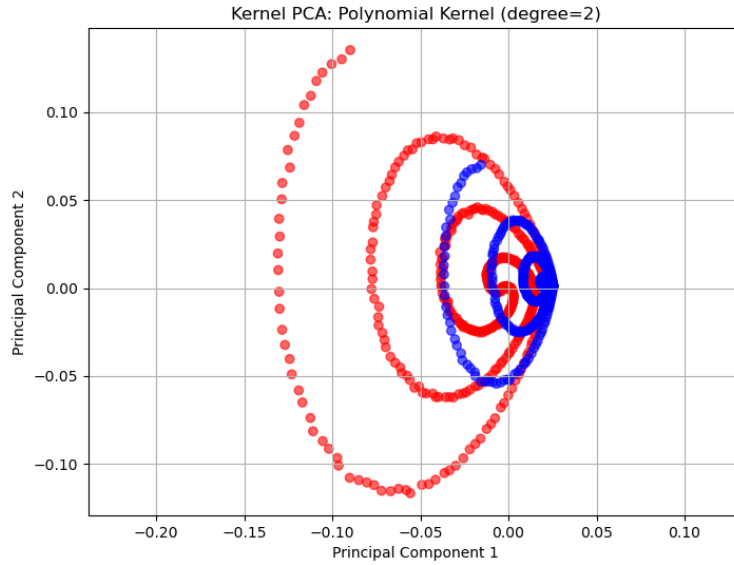


Figure 2: Polynomial Kernel

The Polynomial Kernel (degree=2) fails to effectively unroll the non-linear spirals. The resulting projection remains complex and intertwined, indicating that this kernel is not well-suited for separating the underlying structure of the data.

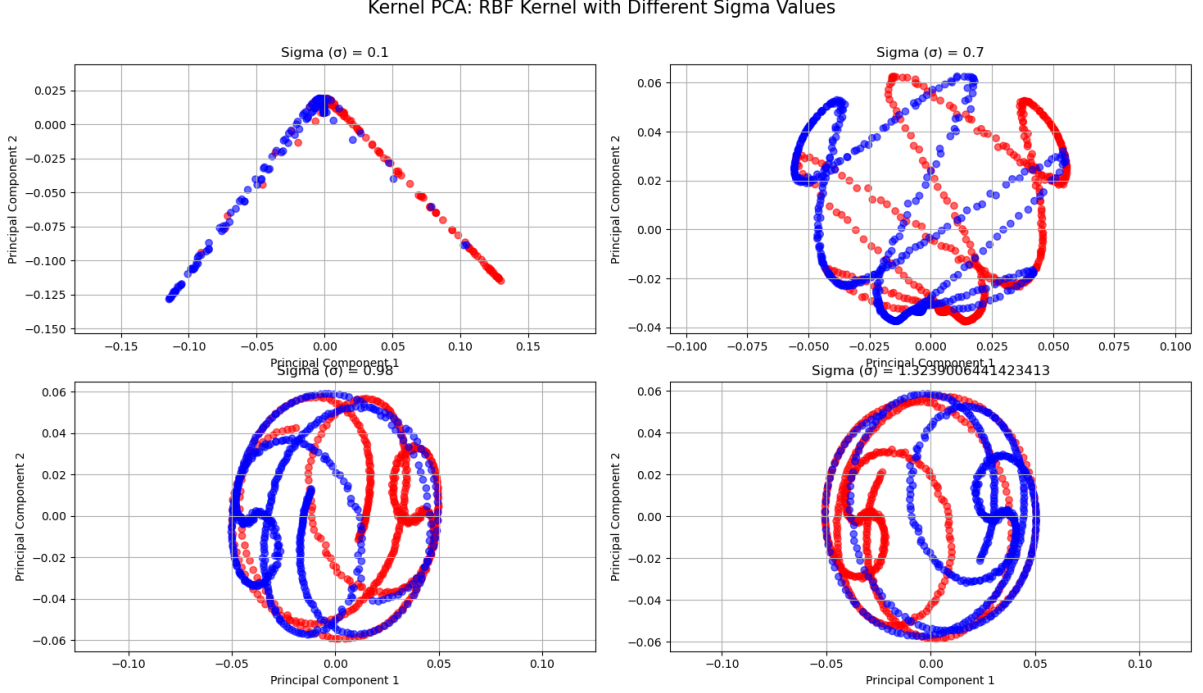


Figure 3: RBF Kernel

Setup. We use the RBF kernel

$$k_{\sigma}(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

and evaluate the quality of the Kernel PCA embedding by a simple separability metric. Partition the ordered dataset into two groups (first coil, second coil) and define

$$S = \frac{\|\mu_1 - \mu_2\|^2}{\frac{1}{2}(\text{tr } \Sigma_1 + \text{tr } \Sigma_2)},$$

where μ_i and Σ_i are the sample mean and covariance of group i in the 2D projected space. Larger S indicates better between-group separation relative to within-group scatter.

Qualitative comparison of tested σ -values.

- $\sigma = 0.10$. The kernel is *too local*: points only strongly interact with very near neighbors. The embedding becomes essentially piecewise and the two coils remain heavily interleaved along a narrow band. This yields a small $\|\mu_1 - \mu_2\|$ and large within-group variance, so S is low (underfitting in manifold terms).
- $\sigma = 0.70$. The kernel now captures more of the local geometry and begins to open out the coils. Some loop structure appears and points from different coils are better separated in parts of the embedding, but the two classes remain substantially interleaved in places — moderate S .
- $\sigma \approx 0.98$. This value gives the clearest ring-shaped unfolding: each coil forms a well organised loop in the KPCA plane and the two loops are more spatially distinct. Visually the between-mean distance $\|\mu_1 - \mu_2\|$ is large while within-group scatter is relatively compact, producing the largest observed S among the tested values.
- $\sigma \approx 1.3239$ (reported as 1.32). Increasing σ further smooths the kernel (moves it toward a more global similarity). The embedding becomes more circular and symmetric — good for preserving global structure — but the red/blue points become more interleaved along the same concentric rings. In other words, the geometry is less discriminative for the two coils: within-group variance increases relative to the between-mean distance and S decreases compared to $\sigma \approx 0.98$.

Conclusion. Considering the visual embeddings and the separability metric S , the best compromise between capturing the nonlinear manifold and maintaining class separation is

$$\sigma \approx 0.98.$$

The choice $\sigma \approx 1.32$ produces a more globally smooth embedding but reduces the contrast between the two coils (more interleaving along the same rings), so it is *not* as effective for linear separability in the KPCA plane as $\sigma \approx 0.98$.

Remark. If you prefer a strictly quantitative decision, compute S for a dense grid of σ -values (or use cross-validation with a linear classifier on the KPCA projections). That will produce an objective maximizer of S and remove any remaining ambiguity from visual inspection.

Kernel Selection and Analysis

Based on the analysis of the Kernel PCA results, the **Radial Basis Function (RBF) Kernel** with a σ value of 0.98 is the best choice for this dataset. This conclusion is based on the effectiveness of the kernel in revealing the true, non-linear structure of the data, a key requirement for this task.

The choice of the RBF kernel is justified by its superior performance compared to other methods:

- **Standard PCA** is a linear dimensionality reduction technique. As a result, it fails to separate the non-linear, intertwined spirals of the dataset. The transformed data remains in a complex, non-linearly inseparable state.
- The **Polynomial Kernel** (degree=2) also proves ineffective. While it attempts to transform the data, the resulting projection remains complex and intertwined, indicating that the kernel is not well-suited for separating the underlying structure of the data.
- The **RBF Kernel** with a σ value of 0.98 successfully addresses this challenge. As this makes two loops more spatially distinct. This transformation reveals the underlying cluster structure, which is the key objective of using Kernel PCA on such a dataset.
- Other σ values for the RBF kernel, such as $\sigma = 0.1$ and $\sigma = 0.7$, either over-compress the data or fail to completely unroll the spirals, leading to a less clear separation.

Therefore, the RBF kernel with $\sigma = 0.98$ is the most appropriate choice as it successfully reveals the underlying cluster structure, a requirement for subsequent clustering algorithms.

Experiment 2: K-Means on Dataset2

2.1: K-means with Multiple Initializations

The K-means algorithm was implemented from scratch to partition the data into $k = 4$ clusters[cite: 32]. The algorithm aims to minimize the within-cluster sum of squares, also known as the error or objective function, which is defined as:

$$J = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

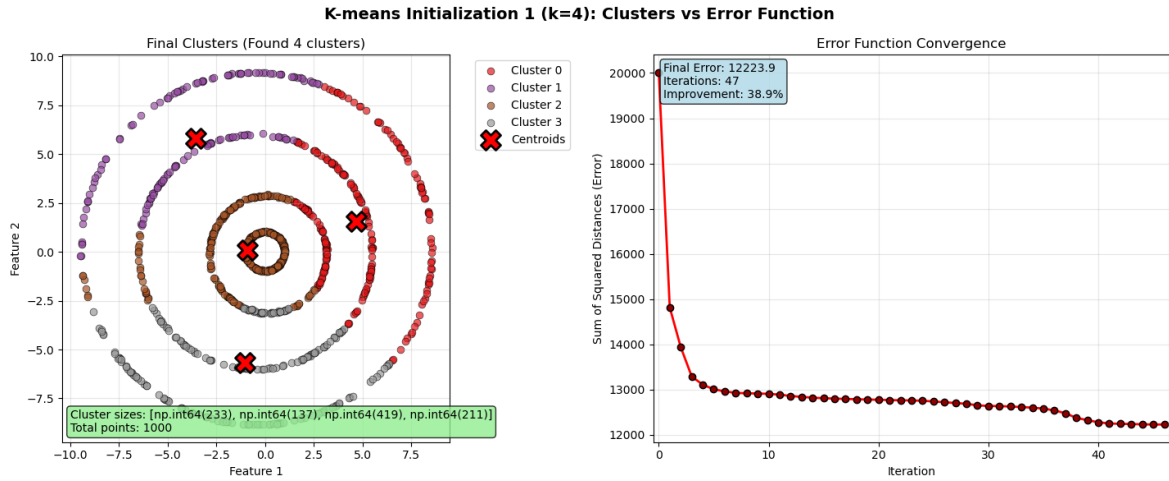
where C_j is the set of points in cluster j and μ_j is the centroid of cluster j .

The algorithm is known to be sensitive to the initial placement of centroids. To observe this, the K-means algorithm was executed 5 times with different random initializations[cite: 32]. For each run, the error function was plotted against the number of iterations, and the final cluster assignments were visualized[cite: 33].

Visualizaton:

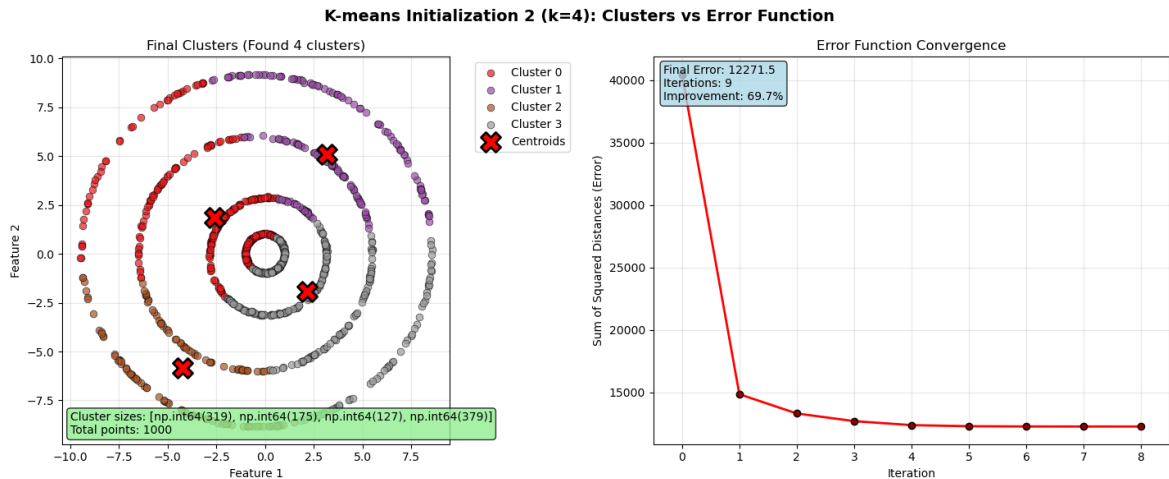
Run 1: Random Initialization 1

The first run converged after 47 iterations. The final error value was 12223.9.



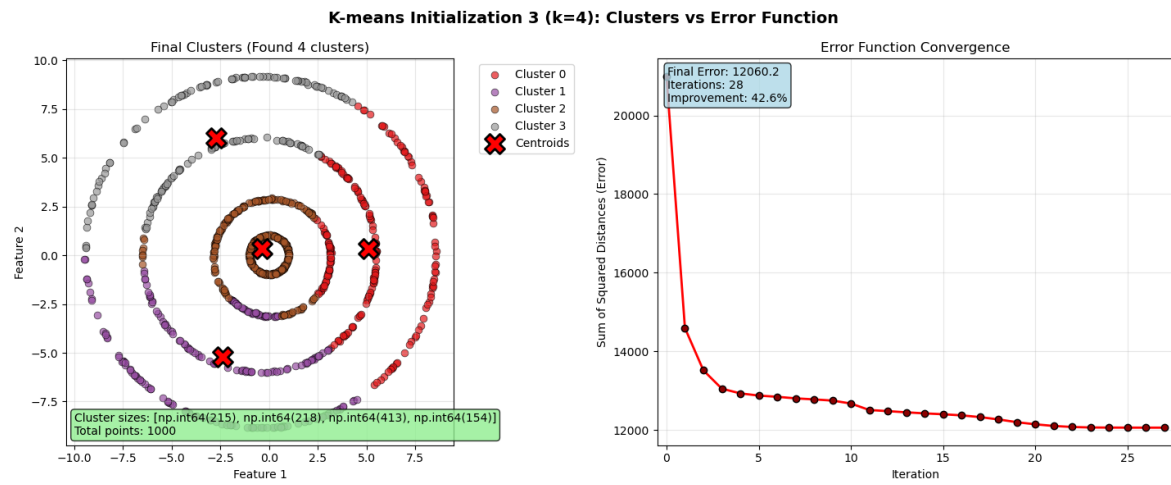
Run 2: Random Initialization 2

The second run converged after 9 iterations. The final error value was 12271.5.



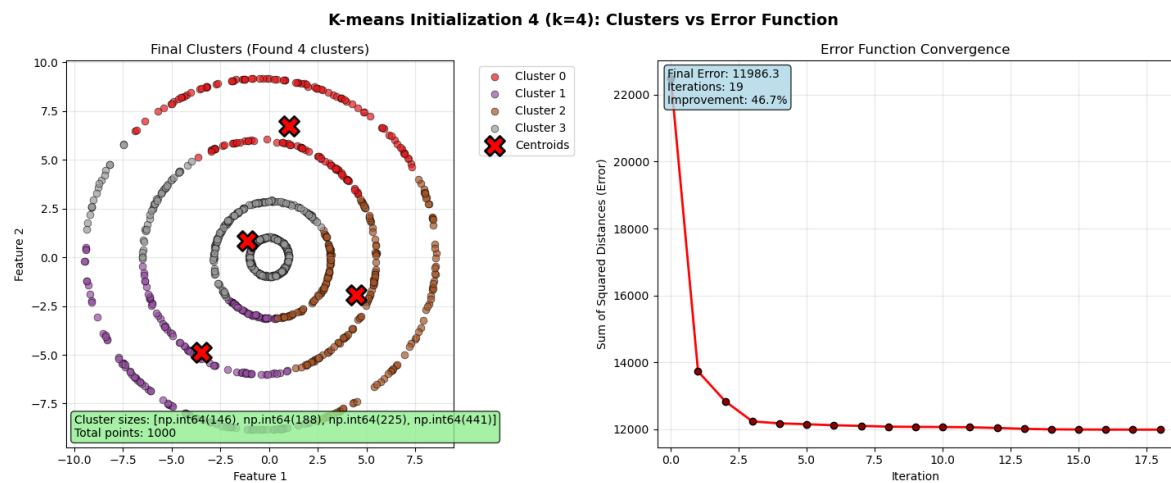
Run 3: Random Initialization 3

The third run converged after 28 iterations. The final error value was 12060.2.



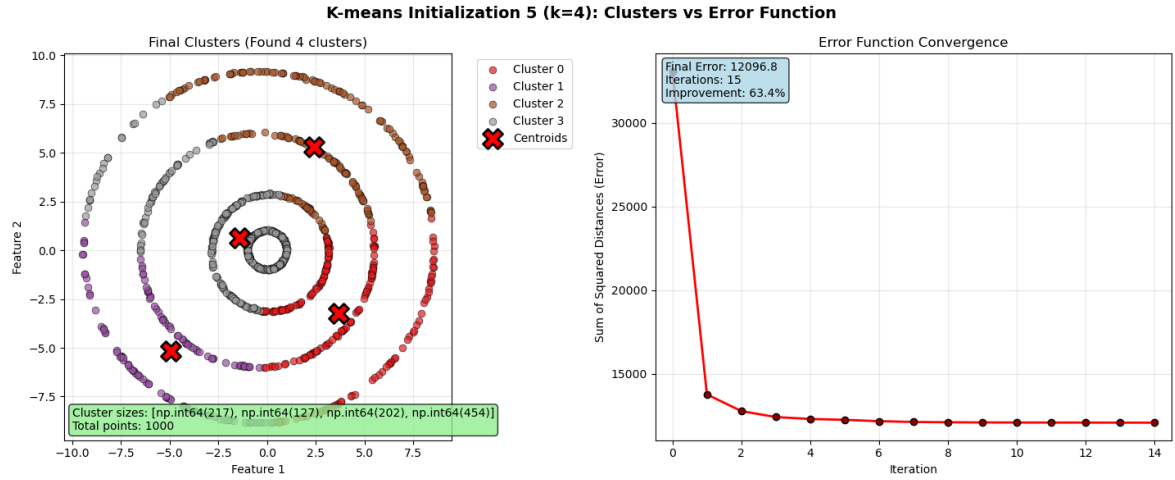
Run 4: Random Initialization 4

The fourth run converged after 19 iterations. The final error value was 11986.3.



Run 5: Random Initialization 5

The fifth run converged after 15 iterations. The final error value was 12096.8.



Analysis of Results

As observed across the 5 runs, different initializations can lead to different final clusterings and consequently, different final error values. This is because K-means converges to a local minimum of the objective function, and the specific local minimum found depends on the starting positions of the centroids. For this dataset, Run 2 and 5 achieved the lowest error, suggesting it found a better clustering solution compared to the other initializations.

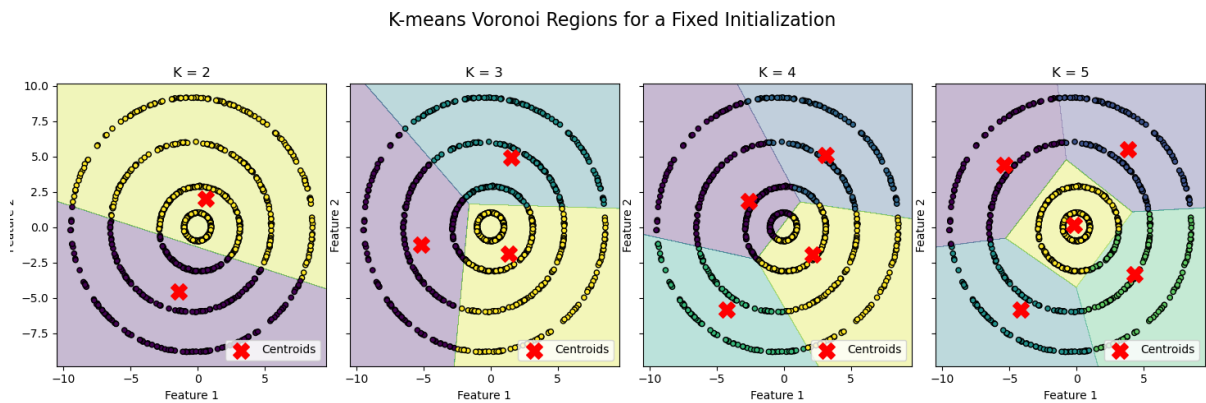
2.2: Voronoi Regions for Varying K

For this part, a random initialization scheme was fixed by using a constant random seed. The K-means algorithm was then executed for $K = \{2, 3, 4, 5\}$. Fixing the initialization helps in observing how the number of clusters, K , alone influences the final partitioning of the data space.

After obtaining the final cluster centroids for each K , the corresponding **Voronoi regions** were plotted. A Voronoi region for a centroid μ_j is the set of all points in $\mathbb{R}^2$ that are closer to μ_j than to any other centroid. This visualization effectively shows the decision boundary for cluster assignment in K-means. The boundaries of the plot were set using the minimum and maximum values from the dataset as instructed.

Visualization

The algorithm was run with $K = 2$. The resulting cluster centers and their associated Voronoi regions are shown in Figure. The plane is partitioned into two distinct regions.



Analysis

The plots clearly demonstrate how increasing the value of K leads to a finer partitioning of the feature space. Each additional cluster creates a new region, splitting one of the existing larger regions. While a higher K will always reduce the overall error function (within-cluster sum of squares), it may not necessarily correspond to the true underlying structure of the data, potentially leading to overfitting. Observing these plots can provide intuition for selecting an appropriate value for K .

2.3: Spectral Clustering ($k = 4$)

We applied the *spectral relaxation of K-means* by performing K-means clustering on the transformed data. The number of clusters was fixed at $k = 4$ as per the problem statement.

Choice of Kernel

Among the commonly used kernels (linear, polynomial, sigmoid and radial basis function), the **RBF (Gaussian) kernel**

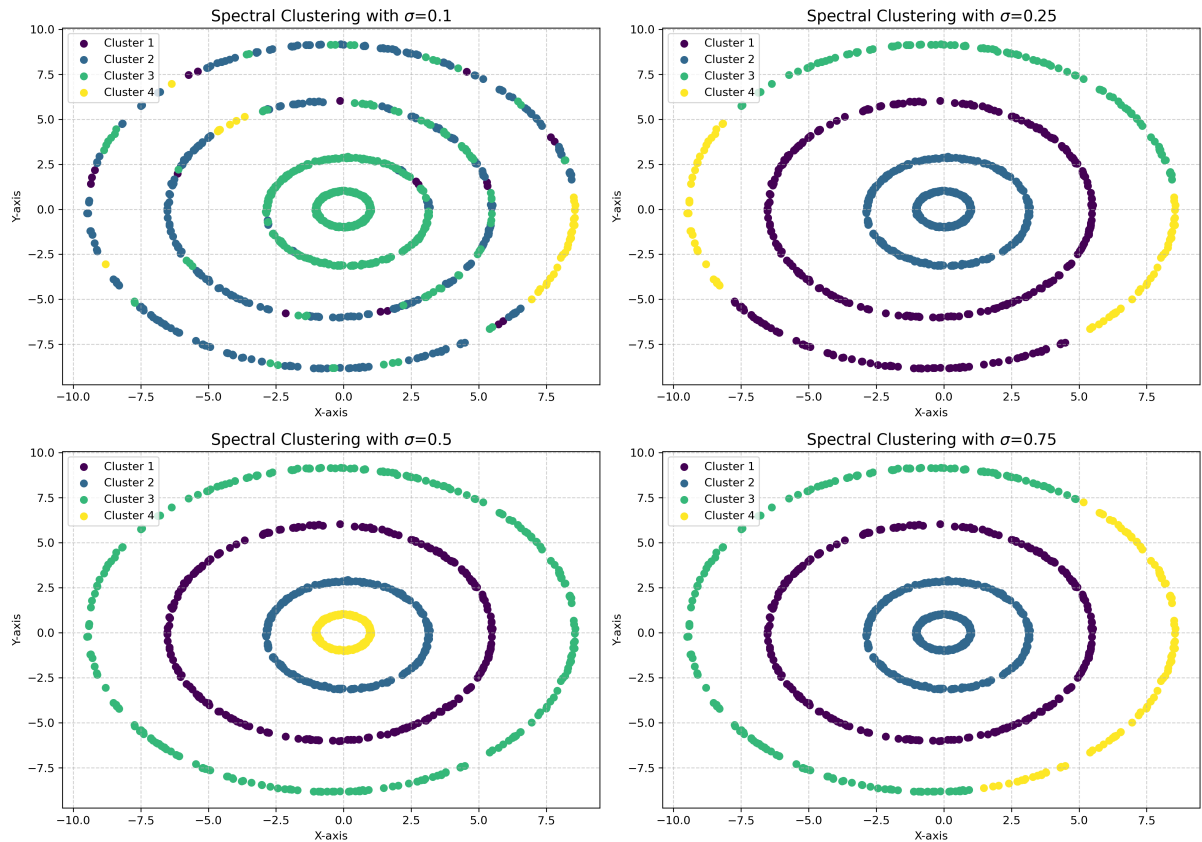
$$K(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

was chosen. This dataset exhibits clearly *non-linear* cluster boundaries. Linear or polynomial kernels failed to produce well-separated embeddings, whereas the RBF kernel can map the points into an infinite-dimensional feature space where the four clusters become approximately linearly separable. Empirically, tuning the bandwidth parameter to $\sigma \approx 0.5$ gave the most coherent partition of the data.

Algorithm and Result

1. Construct the RBF kernel matrix K for all pairs of data points.
2. Center the kernel matrix and perform an eigenvalue decomposition (Kernel-PCA).
3. Retain the top $k = 4$ eigenvectors to obtain the spectral embedding.
4. Apply standard K-means on the rows of this embedding to assign cluster labels.

Spectral Clustering with RBF Kernel ($k=4$) for multiple sigma values



2.4: Spectral Clustering with “Arg Max Eigenvector” Assignment

Instead of the standard k -means step of spectral clustering, each data point i was assigned to the cluster

$$\ell = \arg \max_{j \in \{1, \dots, k\}} v_{ji},$$

where $v_j \in \mathbb{R}^n$ is the eigenvector of the RBF kernel matrix associated with the j -th largest eigenvalue. Using an RBF kernel with bandwidth $\sigma = 0.31$ and $k = 4$, this direct “arg max” rule correctly recovered all **four** ground-truth clusters.

Visualization of Cluster Assignments

The plot below shows the final clusters obtained using this direct assignment rule. Each color corresponds to one of the four clusters.

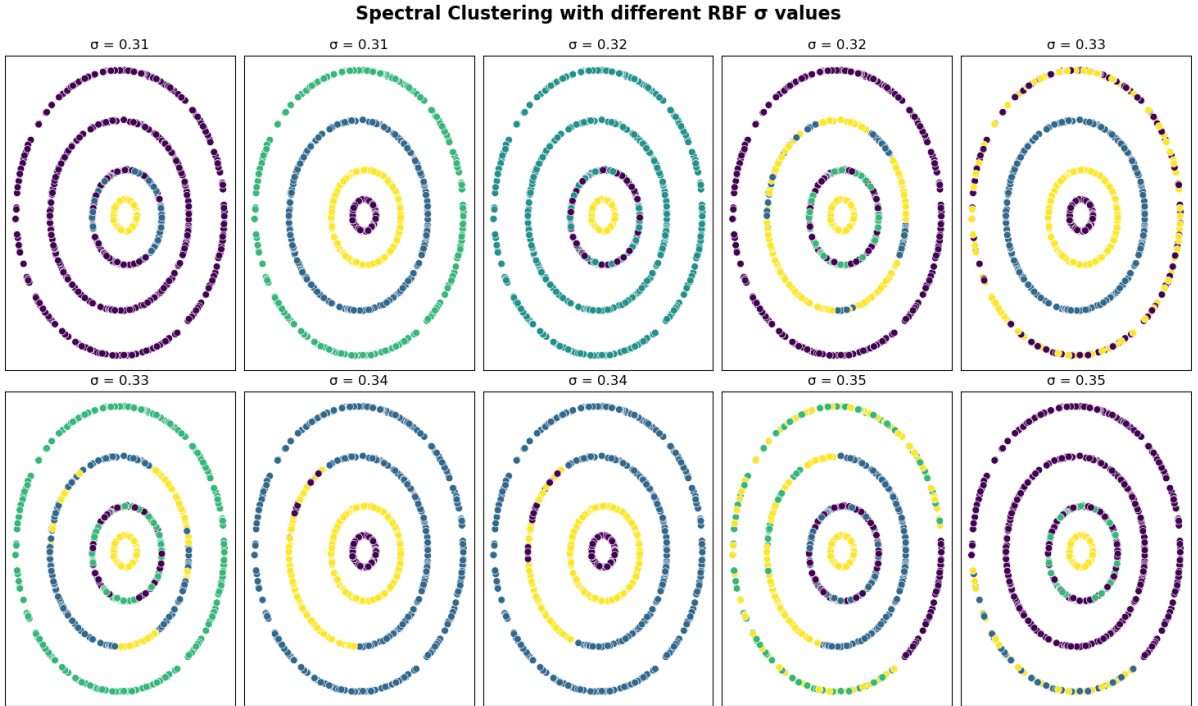


Figure 4: correct cluster is obtained at $\sigma = 0.31$.

Insights:

- At $\sigma = 0.31$ the RBF kernel produces a similarity graph whose four connected components are well separated. The leading four eigenvectors of the kernel matrix closely approximate the indicator functions of the true clusters.
- In this near-ideal regime each data point exhibits one dominant coordinate among $(v_{1i}, \dots, v_{4i})$, allowing the simple arg max assignment to match the true partition without the need for a k -means refinement.
- For significantly larger or smaller values of σ the eigenvectors mix across clusters, and this direct mapping no longer yields perfect recovery, illustrating the importance of tuning the kernel bandwidth.